

DOCUMENTAÇÃO TÉCNICA

Restaurant Management API

Documentação Técnica da API REST

Módulo: Tech Challenge — Fase 01

Autores: André Felix & Ygor Medeiros

Emissão: Maio de 2026

github.com/Andrefelix2810/restaurant-management-api

Introdução

Projeto desenvolvido para o Tech Challenge — Fase 01 da Pós-Tech em Arquitetura e Desenvolvimento Java. A aplicação consiste em um backend REST para gerenciamento de usuários de um sistema compartilhado de restaurantes, contemplando as funcionalidades de cadastro, consulta, atualização, exclusão, troca de senha e validação de login, com persistência em banco relacional PostgreSQL e execução via Docker Compose.

Documento base: Postech - ADJ - Fase 1 - Tech Challenge.pdf

Sumário

1.Objetivo do Projeto	4
2.Tecnologias Utilizadas	6
3.Arquitetura da Solução	6
4.Persistência JDBC e Modelo Relacional	10
5.Funcionalidades Implementadas	12
6.Endpoints da API	13
7.Exemplos de Requisição	14
8.Tratamento de Erros	17
9.Documentação Swagger / OpenAPI	17
10. Collections Postman	18
11. Configuração e Execução	19
12. Docker e Docker Compose	21
13. Build, Testes e Validação	23
14. Decisões Técnicas	24

Objetivo do Projeto

O objetivo principal deste projeto é disponibilizar uma API backend desenvolvida com Spring Boot, estruturada em camadas e preparada para execução em ambiente containerizado.

A aplicação permite realizar operações essenciais sobre usuários, incluindo:

- cadastro de clientes;
- cadastro de donos de restaurante;
- consulta de usuários cadastrados;
- consulta de usuário por identificador;
- atualização de dados cadastrais;
- exclusão de usuários;
- troca de senha;
- validação de login.

A solução utiliza PostgreSQL como banco de dados relacional e Docker Compose para orquestrar a aplicação Java e o serviço de banco de dados, facilitando a reprodução do ambiente em diferentes máquinas.

Tipos de Usuário Suportados

Tipo	Descrição
CLIENT	Representa o cliente que poderá utilizar o sistema.
RESTAURANT_OWNER	Representa o dono de restaurante.

Dados Cadastrais Considerados

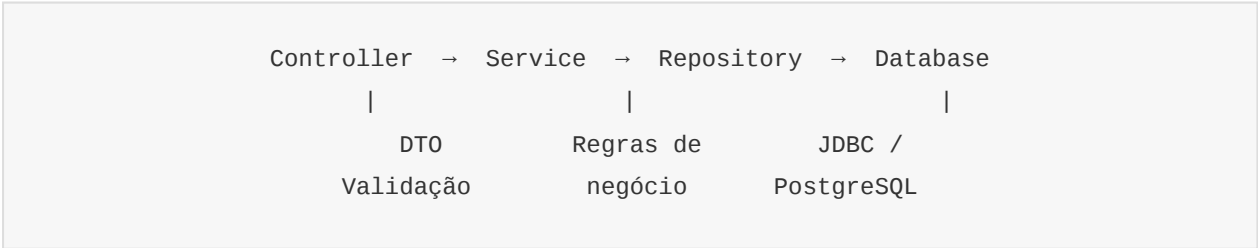
Campo	Tipo	Obrigatório	Observação
name	String	Sim	Nome completo do usuário.
email	String	Sim	Deve possuir formato válido e ser único.
login	String	Sim	Identificador de acesso do usuário e deve ser único.
password	String	Sim	Senha com no mínimo 6 caracteres.
lastModifiedDate	Date/Time	Gerado pelo sistema	Atualizado na criação e alteração do cadastro ou senha.
address	Object	Sim	Endereço completo do usuário.

Tecnologias Utilizadas

Tecnologia	Finalidade
Java 17	Linguagem principal do projeto.
Spring Boot	Framework base para construção da aplicação.
Spring Web MVC	Exposição dos endpoints REST.
Spring JDBC	Persistência e acesso ao banco de dados com SQL explícito.
Jakarta Bean Validation	Validação dos dados de entrada.
PostgreSQL	Banco de dados relacional utilizado pela aplicação.
Maven	Gerenciamento de dependências e build do projeto.
Docker	Criação da imagem da aplicação.
Docker Compose	Orquestração da API e do banco PostgreSQL.
Lombok	Redução de código repetitivo em classes de domínio e DTOs.
Springdoc OpenAPI	Documentação interativa da API via Swagger UI.
Postman	Apoio para testes manuais dos endpoints.

Arquitetura da Solução

A aplicação foi organizada seguindo uma arquitetura em camadas, favorecendo separação de responsabilidades, manutenibilidade e evolução incremental.



Responsabilidade das Camadas

Camada	Responsabilidade
controller	Recebe as requisições HTTP, valida os contratos de entrada e retorna respostas adequadas.
dto.request	Define os objetos de entrada da API, isolando o contrato externo do modelo interno.
dto.response	Define os objetos de resposta devolvidos pela API.
entity	Representa as entidades persistidas no banco de dados.
repository	Realiza a comunicação com o banco por meio do Spring JDBC.
service	Define as operações de negócio disponíveis.
service.impl	Implementa as regras de negócio, validações e orquestração de persistência.
mapper	Converte entidades em DTOs e DTOs em entidades.
exception	Centraliza exceções e padroniza respostas de erro.
config	Agrupar configurações auxiliares, como OpenAPI/Swagger.

Padrões Aplicados

Prática	Aplicação
Separação por camadas	Controllers recebem requisições; services concentram regras; repository executa SQL; DTOs representam contratos externos.
DTOs de entrada e saída	A API não expõe diretamente os objetos internos nem retorna senha nas respostas.
Validação declarativa	Bean Validation com <code>@Valid</code> , <code>@NotBlank</code> , <code>@NotNull</code> , <code>@Email</code> e <code>@Size</code> .
Tratamento centralizado de erro	GlobalExceptionHandler padroniza respostas para validações, regras de negócio, usuário inexistente e erros inesperados.
Persistência explícita	UserRepository usa Spring JDBC com SQL claro para consulta e manipulação dos dados.
Documentação interativa	Springdoc OpenAPI disponibiliza Swagger UI para leitura e teste dos contratos.
Configuração reproduzível	Docker Compose cria o banco PostgreSQL e sobe a aplicação com variáveis de ambiente padronizadas.

Entidade User

Atributo	Descrição
id	Identificador único gerado automaticamente.
name	Nome do usuário.
email	E-mail único do usuário.
login	Login único utilizado para autenticação.
password	Senha do usuário.
lastModifiedDate	Data e hora da última alteração relevante.
role	Tipo do usuário: cliente ou dono de restaurante.
address	Endereço completo associado ao usuário.

Objeto Address

Atributo	Descrição
street	Rua ou avenida.
number	Número do endereço.
neighborhood	Bairro.
city	Cidade.
state	Estado.
zipCode	CEP.
complement	Complemento opcional.

Persistência JDBC e Modelo Relacional

O projeto utiliza Spring JDBC para acesso ao banco de dados. A camada repository trabalha com SQL explícito por meio de `NamedParameterJdbcTemplate`, evitando dependência de JPA/Hibernate e deixando as operações de persistência transparentes.

Tabela `users`

A estrutura da tabela é criada pelo arquivo `src/main/resources/schema.sql` durante a inicialização da aplicação, conforme a propriedade `spring.sql.init.mode`.

Coluna	Tipo	Restrição	Origem no Domínio
<code>id</code>	BIGSERIAL	Chave primária	User.id
<code>name</code>	VARCHAR(255)	Opcional no banco, obrigatório na API	User.name
<code>email</code>	VARCHAR(255)	NOT NULL, UNIQUE	User.email
<code>login</code>	VARCHAR(255)	NOT NULL, UNIQUE	User.login
<code>password</code>	VARCHAR(255)	NOT NULL	User.password
<code>last_modified_date</code>	TIMESTAMP	NOT NULL	User.lastModifiedDate
<code>role</code>	VARCHAR(50)	NOT NULL	User.role
<code>street</code>	VARCHAR(255)	Endereço	Address.street
<code>number</code>	VARCHAR(50)	Endereço	Address.number
<code>neighborhood</code>	VARCHAR(255)	Endereço	Address.neighborhood
<code>city</code>	VARCHAR(255)	Endereço	Address.city
<code>state</code>	VARCHAR(100)	Endereço	Address.state
<code>zip_code</code>	VARCHAR(30)	Endereço	Address.zipCode
<code>complement</code>	VARCHAR(255)	Endereço opcional	Address.complement

Operações Implementadas no Repository

Método	Operação SQL Equivalente	Finalidade
save(User)	INSERT ou UPDATE	Criação e atualização de usuários.
findAll()	SELECT ... FROM users ORDER BY id	Listagem geral.
findById(Long)	SELECT ... WHERE id = :id	Busca por identificador.
findByEmail(String)	SELECT ... WHERE email = :email	Validação de unicidade de e-mail.
findByLogin(String)	SELECT ... WHERE login = :login	Login e validação de unicidade.
existsByEmail(String)	SELECT COUNT(1)	Verifica se e-mail já existe.
existsByLogin(String)	SELECT COUNT(1)	Verifica se login já existe.
delete(User)	DELETE FROM users WHERE id = :id	Exclusão de usuário.

Funcionalidades Implementadas

Gestão de Usuários

- Registro de usuários com perfil de cliente.
- Registro de usuários com perfil de dono de restaurante.
- Listagem de todos os usuários.
- Busca de usuário por ID.
- Atualização de dados cadastrais.
- Exclusão de usuário.
- Atualização automática da data de última modificação.

Validação de Dados

- Validação de campos obrigatórios com `@NotBlank` e `@NotNull`.
- Validação de formato de e-mail com `@Email`.
- Validação de tamanho mínimo de senha com `@Size(min = 6)`.
- Validação de endereço obrigatório.
- Validação de unicidade de e-mail.
- Validação de unicidade de login.

Validação de Login

- Validação de login por meio do endpoint `/auth/login`.
- Verificação de login e senha informados.
- Retorno dos dados do usuário autenticado em caso de sucesso.
- Retorno padronizado de erro para credenciais inválidas.

Troca de Senha

- Verificação da senha antiga antes da alteração.
- Bloqueio de nova senha igual à senha atual.
- Atualização da data de última modificação após alteração.

Endpoints da API

Health Check

Método	Endpoint	Descrição	Status Esperado
GET	/health	Verifica se a API está em execução.	200 OK

Autenticação

Método	Endpoint	Descrição	Status Esperado
POST	/auth/login	Valida login e senha do usuário.	200 OK

Usuários

Método	Endpoint	Descrição	Status Esperado
POST	/users/register/client	Cadastra um usuário com perfil CLIENT.	201 Created
POST	/users/register/owner	Cadastra um usuário com perfil RESTAURANT_OWNER.	201 Created
GET	/users	Lista todos os usuários cadastrados.	200 OK
GET	/users/{id}	Busca um usuário pelo ID.	200 OK
PUT	/users/{id}	Atualiza os dados cadastrais de um usuário.	200 OK
PATCH	/users/{id}/password	Altera a senha de um usuário.	204 No Content
DELETE	/users/{id}	Remove um usuário pelo ID.	204 No Content

Exemplos de Requisição

Registrar Cliente

```
POST /users/register/client
Content-Type: application/json

{
  "name": "Luis Ricardo",
  "email": "luis.ricardo@email.com",
  "login": "luisricardo",
  "password": "senha123",
  "address": {
    "street": "Avenida Paulista",
    "number": "1000",
    "neighborhood": "Bela Vista",
    "city": "São Paulo",
    "state": "SP",
    "zipCode": "01310100",
    "complement": "Bloco B, Apto 42"
  }
}
```

Resposta Esperada

```
{
  "id": 1,
  "name": "Luis Ricardo",
  "email": "luis.ricardo@email.com",
  "login": "luisricardo",
  "lastModifiedDate": "2026-05-02T10:00:00",
  "role": "CLIENT",
  "address": {
    "street": "Avenida Paulista",
    "number": "1000",
    "neighborhood": "Bela Vista",
    "city": "São Paulo",
    "state": "SP",
    "zipCode": "01310100",
    "complement": "Bloco B, Apto 42"
  }
}
```

Registrar Dono de Restaurante

POST /users/register/owner

Content-Type: application/json

```
{
  "name": "Mariana Souza",
  "email": "mariana.souza@email.com",
  "login": "marianasouza",
  "password": "senha123",
  "address": {
    "street": "Rua Augusta",
    "number": "500",
    "neighborhood": "Consolação",
    "city": "São Paulo",
    "state": "SP",
    "zipCode": "01305000",
    "complement": "Sala 10"
  }
}
```

Realizar Login

POST /auth/login

Content-Type: application/json

```
{
  "login": "luisricardo",
  "password": "senha123"
}
```

Atualizar Usuário

```
PUT /users/1
Content-Type: application/json

{
  "name": "Luis Ricardo Almeida",
  "email": "luis.almeida@email.com",
  "login": "luisalmeida",
  "address": {
    "street": "Rua Vergueiro",
    "number": "2000",
    "neighborhood": "Vila Mariana",
    "city": "São Paulo",
    "state": "SP",
    "zipCode": "04101000",
    "complement": "Casa 2"
  }
}
```

Alterar Senha

```
PATCH /users/1/password
Content-Type: application/json

{
  "oldPassword": "senha123",
  "newPassword": "novaSenha123"
}
```

Excluir Usuário

```
DELETE /users/1
```

Resposta esperada: 204 No Content

Tratamento de Erros

O projeto possui tratamento global de exceções com `@RestControllerAdvice`, garantindo respostas padronizadas para erros de validação, regras de negócio e recursos inexistentes.

Formato Padrão de Erro

```
{
  "timestamp": "2026-05-02T10:30:00",
  "status": 400,
  "error": "Bad Request",
  "message": "Email already registered",
  "path": "/users/register/client"
}
```

Principais Cenários Tratados

Cenário	Exceção / Validação	Status HTTP
Usuário inexistente	UserNotFoundException	404 Not Found
E-mail em formato inválido	Bean Validation	400 Bad Request
Campo obrigatório ausente	Bean Validation	400 Bad Request
E-mail já cadastrado	BusinessException	400 Bad Request
Login já cadastrado	BusinessException	400 Bad Request
Senha antiga incorreta	BusinessException	400 Bad Request
Credenciais inválidas	BusinessException	400 Bad Request
Erro não previsto	Exception	500 Internal Server Error

Documentação Swagger / OpenAPI

A aplicação possui documentação interativa por meio do Springdoc OpenAPI. Após iniciar a aplicação, acesse:

Swagger UI: <http://localhost:8080/swagger-ui/index.html>

Especificação JSON: <http://localhost:8080/v3/api-docs>

A documentação Swagger permite visualizar os endpoints disponíveis, contratos de entrada, respostas esperadas e realizar testes diretamente pelo navegador.

SEÇÃO 10

Collections Postman

O projeto possui uma collection Postman para apoiar os testes manuais dos endpoints da API.

Localização: `postman/Restaurant-API-Local.postman_collection.json`

Configuração e Execução

Pré-requisitos

Para execução local sem Docker, é necessário possuir:

- Java 17 instalado;
- Maven instalado ou uso do Maven Wrapper do projeto;
- PostgreSQL instalado e em execução;
- banco de dados `restaurant_db` criado.

Configuração Padrão

O arquivo `src/main/resources/application.properties` utiliza variáveis de ambiente com valores padrão:

```
spring.application.name=restaurant-management-api
server.port=8080
spring.datasource.url=${SPRING_DATASOURCE_URL:jdbc:postgresql://localhost:5432/restaurant_db}
spring.datasource.username=${SPRING_DATASOURCE_USERNAME:postgres}
spring.datasource.password=${SPRING_DATASOURCE_PASSWORD:postgres}
spring.sql.init.mode=${SPRING_SQL_INIT_MODE:always}
```

Executar Localmente com Maven

No diretório raiz do projeto, execute:

```
mvn spring-boot:run
```

Ou, utilizando o Maven Wrapper:

```
# Windows
mvnw.cmd spring-boot:run

# Linux / macOS
./mvnw spring-boot:run
```

Verificar se a Aplicação Subiu

```
curl http://localhost:8080/health
```

Resposta esperada: API is running

Docker e Docker Compose

A aplicação está preparada para execução containerizada, utilizando um container para o backend Java e outro para o PostgreSQL.

Serviços Configurados

Serviço	Imagem / Base	Porta	Descrição
postgres	postgres:16-alpine	5432	Banco de dados relacional da aplicação.
app	Build local via Dockerfile	8080	API Spring Boot.

Executar com Docker Compose

```
# Subir todos os serviços
docker compose up --build

# Parar os containers
docker compose down
```

Variáveis de Ambiente Suportadas

Variável	Valor Padrão	Descrição
DB_NAME	restaurant_db	Nome do banco de dados criado no PostgreSQL.
DB_USER	postgres	Usuário do banco.
DB_PASSWORD	postgres	Senha do banco.
SPRING_DATASOURCE_URL	jdbc:postgresql://localhost:5432/restaurant_db	URL de conexão da aplicação.
SPRING_DATASOURCE_USERNAME	postgres	Usuário utilizado pela aplicação.
SPRING_DATASOURCE_PASSWORD	postgres	Senha utilizada pela aplicação.
SPRING_SQL_INIT_MODE	always	Controla a execução do schema.sql na inicialização.

Build, Testes e Validação

Compilar o Projeto

```
# Windows
mvnw.cmd -DskipTests compile

# Linux / macOS
./mvnw -DskipTests compile
```

Executar Testes Automatizados

```
# Windows
mvnw.cmd test

# Linux / macOS
./mvnw test
```

Validação Manual com Postman

A validação funcional recomendada para a Fase 01 deve utilizar a collection Postman disponível em:

```
postman/Restaurant-API-Local.postman_collection.json
```

Essa collection permite testar os fluxos exigidos no PDF: cadastro, consulta, alteração, exclusão, troca de senha e validação de login.

Decisões Técnicas

| Separação entre DTO e Entidade

O projeto utiliza DTOs para entrada e saída de dados, evitando expor diretamente os objetos de domínio nos contratos da API. Essa abordagem facilita a manutenção, melhora o controle sobre os dados trafegados e reduz o acoplamento entre a camada externa e o modelo de persistência.

| Uso de Camada de Serviço

As regras de negócio foram centralizadas em `UserServiceImpl`, mantendo os controllers mais simples e focados apenas na comunicação HTTP. Essa separação favorece testes, reuso e evolução da aplicação.

| Uso de Mapper

A classe `UserMapper` concentra a conversão entre DTOs e entidades, evitando duplicação de código e mantendo a responsabilidade de transformação isolada.

| Tratamento Global de Exceções

O uso de `GlobalExceptionHandler` permite padronizar as respostas de erro da API e melhorar a experiência de consumo dos endpoints.

| Execução Containerizada

O Docker Compose permite subir aplicação e banco de dados com um único comando, reduzindo problemas de configuração local e garantindo maior previsibilidade no ambiente de execução.

Possíveis Evoluções Futuras

Embora o projeto atenda ao escopo da Fase 01, algumas melhorias podem ser consideradas para fases posteriores:

- implementação de autenticação com JWT;
- criptografia de senha com BCrypt;
- inclusão de Spring Security;
- criação das entidades de restaurante, cardápio, pedido e avaliação;
- implementação de testes unitários e de integração mais abrangentes;
- inclusão de migrations com Flyway ou Liquibase;
- paginação e filtros na listagem de usuários;
- uso de perfis de ambiente, como dev, test e prod;
- implantação em ambiente cloud;
- melhoria da collection Postman com variáveis dinâmicas e scripts de teste.

Apresentando uma API REST funcional, estruturada em camadas, integrada a banco relacional, documentada, testável via Postman e executável por Docker Compose. A solução foi construída com foco em organização, clareza arquitetural e possibilidade de evolução, permitindo que novas funcionalidades do domínio de restaurantes sejam incorporadas nas próximas fases do projeto sem comprometer a estrutura atual.